

INF2006

CLOUD COMPUTING & BIG DATA

TEAM PROJECT 1

Secure, Scalable and Data-Driven Cloud Service

Project Information	Details
Group	G014
Project Title	Airbnb Pricing and Market Intelligence Platform
Cloud Provider	AWS
Team Members	Lum Jia Jun (2500022), Nixon Lee Disheng (2500594), Madugula Adheesh (2500670), Leow Yi Hao Ignatius (2501538), Wong Zhen Ho Brendan (2503427)
Submission	Sunday, 11 October, 11:59 PM

INF2006 | Team Project 1 | [Airbnb Pricing and Market Intelligence Platform]

1. Problem, Users and Success Criteria

Airbnb hosts and prospective hosts must choose a nightly price in markets that vary by location, room type, property characteristics, capacity, host characteristics, and review history. A single global average is therefore not useful, particularly because the supplied dataset covers ten cities with prices recorded in each city's local currency. Users also need to understand the surrounding historical market rather than treating a model output as an objectively correct price.

The primary user is a host or prospective host exploring a listing configuration. The main workflow accepts supported listing attributes, validates them in both the browser and backend, runs a trained regression pipeline, and returns an estimated nightly price in the relevant local currency with an explicit disclaimer. Signed-in users can save their estimates and later retrieve only their own prediction history. A public market view provides city-level listing counts, median prices, and ratings from the processed analytical dataset.

The project defines success as a working, reproducible workflow rather than maximum feature coverage. Functional success requires a CloudFront-hosted frontend, a healthy API, model inference, ten-city analytics, authenticated prediction history and AI/chat. Data success requires documented profiling, reproducible preprocessing and training, held-out evaluation, and a cloud pipeline that produces queryable Parquet data. Security success requires private S3 origins, HTTPS, least-privilege IAM roles, Cognito JWT authorisation, input validation and encryption at rest. Operational success requires structured logs, metrics, alarms, recovery controls and appropriately qualified load-test results.

The service is deliberately scoped. Predictions cover the typical market up to a city-specific 99th-percentile boundary learned from the training data. They are not valuations, guarantees, or financial advice. Analytics values use different local currencies and therefore should not be compared as though they were denominated in the same currency. Success does not mean eliminating model error, supporting unlimited traffic, or proving production readiness from a single university development account.

2. Solution Overview and Architecture

2.1 Solution Overview

The **Airbnb Pricing & Market Intelligence Platform** helps Airbnb hosts and prospective hosts choose a reasonable nightly price in a multi-city, mixed-currency market. It combines a trained price-estimation model with city-level market analytics and protected user prediction history.

Users can access the pricing estimator and market dashboard publicly. Authenticated users can save predictions and retrieve only their own history. The platform also provides an authenticated AI/chat feature that uses relevant user prediction information. Its Claude Haiku call is capped at 300 output tokens, while the displayed plain-text response is bounded to 120 words. Predictions are presented as estimates rather than guaranteed market prices, with model and dataset limitations documented in the project.

The solution is a serverless AWS application using **CloudFront, API Gateway, Lambda, Cognito, DynamoDB, S3, Glue and Athena**, with Amazon Bedrock providing the AI/chat model. Public routes support health checks, prediction and analytics, while authenticated routes provide history and AI/chat functionality.

2.2 Architecture Diagram

The platform is deployed on AWS in the ap-southeast-1 Singapore region and separates public-facing services from protected application and data resources. The live CloudFront URL as of 7 October 2026 is <https://dqvmcy3nxs3v.cloudfront.net> and remains valid unless the stack is destroyed and recreated.

- g. The history function retrieves only records belonging to the authenticated user.
- h. Separately, raw Airbnb data is stored in S3, transformed into Parquet by Glue and queried using Athena for market analytics.
- i. The AI/chat route uses authenticated, relevant prediction information before invoking the selected AI model.
- j. CloudWatch collects application logs and metrics, with SNS providing configured alerts.

Security note: User identity is derived from validated Cognito JWT claims rather than caller-provided IDs, supporting user-level isolation.

2.4 Architecture Rationale

The main architecture decisions are summarised below.

- Serverless services suit the low-to-medium, request-driven workload and reduce idle infrastructure and server maintenance.
- Private S3 with CloudFront provides cached HTTPS delivery while preventing direct bucket access; cache invalidation is the main trade-off.
- DynamoDB matches the per-user history access pattern without requiring relational-database infrastructure.
- S3, Glue and Athena separate analytics from transactional data and provide a reproducible processing workflow.
- Authenticated AI/chat access is limited to relevant prediction information, reducing exposure and unnecessary usage.
- Private networking, security groups and VPC endpoints provide controlled backend connectivity.

Overall, the architecture prioritises security, maintainability and reproducibility. Its main trade-offs are AWS service dependence, cold starts, quotas, and fixed hourly charges for interface VPC endpoints.

3. Cloud Service/Deployment Choices and Trade-offs

3.1 Cloud Service Model

The solution is primarily serverless and uses managed AWS services for compute, API management, storage, analytics and AI. Its limited IaaS-style footprint comprises the VPC and related network controls.

AWS Lambda, Amazon API Gateway, AWS Glue and Amazon Athena provide managed application execution and data processing. Terraform provisions the VPC, subnets, route tables, security groups and VPC endpoints used for controlled connectivity.

3.2 Deployment Model

The platform uses AWS public cloud in ap-southeast-1 Singapore, with Terraform for Infrastructure as Code.

The current architecture uses VPC endpoints for controlled AWS service connectivity and does not deploy a NAT Gateway.

3.3 Selected Cloud Services

The table below summarises the AWS services used and their purpose.

AWS service	Purpose
Amazon S3	Stores the private frontend and processed Airbnb datasets
Amazon CloudFront	Provides HTTPS delivery, caching and frontend distribution
AWS WAF	Provides edge protection and rate-based controls
API Gateway	Provides the HTTP API and routes requests to Lambda

AWS Lambda	Runs the core application logic for health checks, prediction, analytics, history and AI/chat functionality
Amazon Cognito	Provides user registration, sign-in and JWT-based authorisation
Amazon DynamoDB	Stores authenticated user's prediction history
AWS Glue	Performs ETL operations, including cleaning and transforming the dataset into curated data
AWS Glue Data Catalog	Stores metadata and schema information for processed dataset
Amazon Athena	SQL-based analytics on the processed Airbnb dataset
Amazon Bedrock	Provides the Claude-based LLM capability for the platform chatbot
Amazon Elastic Container Registry (ECR)	Stores the container image for the prediction Lambda; the ML stack and model bundle are too large for a ZIP deployment.
Amazon Virtual Private Cloud (VPC)	Provides network isolation for the backend components
Amazon CloudWatch	Provides logging, metrics, dashboards and operational monitoring
Amazon SNS	Operational alert notifications
AWS KMS	Encrypts selected resources, including the SNS topic used for operational alerts
HashiCorp Terraform	Provision and manage the AWS infrastructure as code

Together, these services provide the required compute, storage, networking, authentication, analytics, AI, security and monitoring capabilities without requiring traditional server infrastructure.

3.4 Alternatives Considered

EC2-Based Web Hosting

An alternative was an **EC2-based application using a web server and Auto Scaling Group**. While this would provide greater control over the operating system and runtime, it would also introduce server maintenance, patching and idle infrastructure costs.

API Gateway and Lambda were selected because the expected university-scale workload is low-volume and uneven.

Relational Database and Application Server

A relational database with an application server was considered. DynamoDB handles user history, while S3, Glue and Athena handle analytics.

DynamoDB suits the narrow per-user history access pattern, while Athena provides analytical querying without requiring a continuously running database server. This reduces infrastructure complexity and operational overhead.

Traditional VPC-Heavy Architecture

A more traditional infrastructure design with extensive server-based networking was also considered. The current architecture instead uses private subnets, security groups and VPC endpoints where required, maintaining a smaller infrastructure footprint while still providing network isolation and controlled access.

3.5 Scalability, Reliability and Cost Assumptions

The architecture targets a university-scale, low-to-medium, bursty request workload and avoids permanently running application servers.

DynamoDB supports variable prediction-history demand, while S3/Glue/Athena separate analytical processing.

CloudWatch/SNS provide monitoring and alerts; DynamoDB point-in-time recovery and S3 protection support data protection.

Testing demonstrates behaviour only for the measured workloads, not unlimited scalability.

Serverless compute, Parquet and query/retention controls help manage variable costs, while interface VPC endpoints introduce fixed hourly networking charges even at low request volume.

4. Implementation, Data Design and Security Controls

4.1 Application Implementation

The application uses a CloudFront frontend with API Gateway and focused Lambda routes. Users submit validated listing information for prediction; authenticated users can manage their own history, while analytics provides fixed city-level summaries.

The application is separated into focused Lambda functions for health checks, price prediction, analytics, history, and AI/chat. The prediction function performs inference using the exported model pipeline, while the analytics function executes a fixed Athena query rather than accepting arbitrary SQL. The history function retrieves only the authenticated user's records, and the chat function uses bounded prompts and outputs. This separation makes the functions easier to maintain, audit and secure.

4.2 Data Model and Storage

The platform separates operational user data from analytical data.

DynamoDB stores prediction history using the verified Cognito subject as the user partition key. A timestamp and prediction identifier are used to support retrieval of recent predictions without requiring expensive table scans. This structure matches the application's main access pattern: retrieving predictions belonging to the logged-in user.

The analytical dataset is stored in a private **S3 data lake**. Raw Airbnb listings are retained under a raw prefix and transformed by **AWS Glue** into city-partitioned Parquet files. **Amazon Athena** queries the processed data using a controlled workgroup with scan limits and result expiry.

4.3 Input Validation

Input validation is performed at both the frontend and backend. Client-side validation checks basic conditions before submission, while the prediction API performs stricter validation before invoking the model.

The prediction Lambda rejects missing fields, unsupported categories, invalid city-neighbourhood combinations, non-finite values, invalid coordinates and unreasonable numeric inputs. Invalid requests return controlled JSON errors rather than exposing internal stack traces.

The AI route also applies prompt and output limits to reduce abuse and unnecessary processing costs. These controls ensure that malformed or excessive input does not reach the model unchecked.

4.4 Authentication and Authorisation

Authentication uses Amazon Cognito with the authorization-code flow and PKCE. API Gateway validates JWTs on protected routes, and the public frontend client does not store a client secret. The report does not claim a specific MFA method because the corresponding configuration evidence was not available for this review.

API Gateway validates JWTs for protected routes. The backend does not trust a caller-provided user ID; instead, it derives the user's identity from the verified **Cognito subject claim**.

The history function queries records using the verified Cognito subject, and the chat function accesses only that user's recent predictions and the permitted AI model. Synthetic owner and non-owner identities validated the application-level partition check. A full two-browser test using two separate Cognito accounts was not performed.

4.5 Secrets and Network Security

The project uses a least-privilege approach for credentials and network access. AWS credentials are not stored in Git, while Lambda uses IAM execution roles instead of embedded access keys. The deployment workflow uses **GitHub OIDC** rather than long-lived AWS credentials. Sensitive files such as environment files, Terraform state, and private keys are excluded from the repository.

Network and data protections include:

- S3 Block Public Access for stored objects.
- CloudFront Origin Access Control for the private frontend bucket.
- CORS restricted to the deployed CloudFront origin.
- HTTPS-only browser and CloudFront communication.
- CloudFront security headers including CSP, HSTS and anti-framing protection.
- Private Lambda subnets across two availability zones.
- Gateway and interface VPC endpoints for controlled AWS service access.
- Encryption at rest for relevant storage and logs.

These controls reduce direct exposure of application resources while limiting access to the services and users that require it.

4.6 Threat-Control Mapping

A key threat is unauthorised prediction-history access. Cognito JWT validation, the verified subject claim and application-level authorisation prevent callers from selecting another user's DynamoDB partition. Synthetic owner and non-owner identities validated this control, but a full two-account browser test remains outstanding.

A second threat is **direct public access to frontend S3 objects**. S3 Block Public Access and a bucket policy restrict object access to the CloudFront distribution. Security testing confirms that direct S3 object requests are rejected while the CloudFront distribution remains accessible.

Threat	Primary Control
Unauthorised prediction-history access	Cognito JWT validation + verified subject claim
Direct S3 object access	S3 Block Public Access + CloudFront OAC
AI cost abuse	Authentication, API throttling and bounded prompts/outputs
OAuth interception	Authorization Code + PKCE
Unencrypted browser traffic	HTTPS enforcement
Sensitive prompt logging	Controlled logging and prompt handling

5. Analytics or AI/ML Feature: Data, Method, Evaluation and Limitations

5.1 Purpose and Workflow Integration

The analytics/ML capability provides price estimates alongside city-level market context.

The browser submits listing attributes to the prediction endpoint, which returns an estimated price with a disclaimer. Analytics provides listing counts, median prices and ratings for broader market context.

5.2 Dataset and Provenance

The project uses the **Kaggle Airbnb Listings & Reviews dataset**, attributed to **mysarahmadbhat** and released under **CC0**. The profiled dataset contains **279,712 listings and 5,373,143 reviews across ten cities**. The review data contains identifiers and dates but no review text.

The absence of review text is important because it prevents unsupported sentiment analysis. The project therefore does not fabricate sentiment features and instead focuses on the listing attributes supported by the dataset.

5.3 Features and Preprocessing

The model uses pricing-related listing attributes including **city, neighbourhood, coordinates, room type, property type, capacity, bedrooms, minimum nights, rating, host listing count, amenity count, instant bookability, superhost status and identity verification**. The target variable is nightly price.

Preprocessing includes:

1. Removing invalid and non-positive prices.
2. Median imputation for missing numerical values.
3. Categorical encoding for the selected model.
4. A city-stratified 80/20 train/test split.
5. A log transformation of the target to address its strong right-skew.
6. City-specific 99th-percentile price limits learned from the training data to reduce the influence of extreme luxury listings.
7. Exclusion of unsupported identifiers and free-text fields.

5.4 Method

Three approaches were compared: a global median baseline, a regularised linear model (Ridge), and a Histogram Gradient Boosting Regressor. Models were evaluated on the held-out test split using median city-normalised MAE as the primary selection metric. For each city, normalised MAE is calculated as that city's MAE divided by the median observed test-set price for that city; the final metric is the median of these per-city values. This prevents a city with a larger currency denomination from dominating model selection. The same held-out split was used both to select the model and to report the final metrics, with no separate untouched final test set, so the reported figures may be slightly optimistic.

The reported median city-normalised MAE values were 1.148 for the global median baseline, 0.682 for Ridge regression and 0.592 for Histogram Gradient Boosting, which was selected as the final model. A value of 0.592 means that, for the median city, MAE was 59.2% of that city's median observed test price. It is not an accuracy percentage and indicates substantial residual error.

5.5 Evaluation

The selected Histogram Gradient Boosting model achieved the following results on the supported held-out scope:

Metric	Result
MAE	191.836
RMSE	629.762
R ²	0.644
Log-price R ²	0.850
Median city-normalised MAE (selection metric)	0.592

The median city-normalised MAE of 0.592 indicates substantial error relative to city median prices. Log-price R² of 0.850 measures explained variation after transforming the target and is not directly

comparable with normalised MAE. The model is suitable for decision support within the stated scope, not for high-precision valuation.

The analytics pipeline was also validated separately. AWS Glue processed the listing data in approximately **90 seconds**, while Athena returned summaries for all ten cities after scanning **566,077 bytes**. This demonstrates that the analytics workflow operates as intended under the tested conditions.

5.6 Limitations and Responsible Use

The model has several important limitations:

- The dataset is **cross-sectional rather than time-series**, so it cannot forecast future price movements.
- Cities use different local currencies, limiting direct cross-city price comparisons.
- Missing values are imputed, introducing assumptions into the model.
- The top 1% of prices is excluded from the supported scope, limiting representation of luxury listings.
- Historical listing data may not reflect current local market conditions.
- Model predictions should not be treated as guaranteed prices, valuations or financial recommendations.

5.7 Reproducibility

The model development process can be reproduced using the project's documented scripts:

```
pip install -r requirements.txt
python profile_data.py --listings "../data/raw/Airbnb Data/Listings.csv" --reviews "../data/raw/Airbnb Data/Reviews.csv" --output artifacts/data_profile.json
python train_model.py --listings "../data/raw/Airbnb Data/Listings.csv" --model-output artifacts/airbnb_price_model.joblib --metrics-output artifacts/model_evaluation.json
```

The process profiles the dataset, trains the model, exports the preprocessing and model pipeline, and produces the evaluation metrics. Generated model artefacts are kept outside version control, while the documented scripts and evaluation provide a reproducible development process.

6. Testing, Scalability/Resilience and Monitoring Results

6.1 Test Strategy

The project used a layered testing strategy covering functional behaviour, security controls, data/AI validation, resilience, and operational monitoring. Testing combined live HTTP checks and controlled cloud tests to verify the deployed user workflow.

6.2 Functional Workflow Test

The functional test verified the deployed frontend, health endpoint, pricing prediction and request validation.

Setup: Terraform-managed AWS environment in ap-southeast-1, with CloudFront, API Gateway and Lambda.

Test: The CloudFront frontend and health endpoint were accessed, followed by the API smoke test using valid and malformed requests.

Expected: Frontend and health requests return HTTP 200, valid listing data produces a numeric prediction, and malformed input is rejected.

Result: Passed. The frontend and health endpoint returned HTTP 200, the health response contained the expected status values, a valid request returned a numeric EUR prediction, and malformed JSON returned HTTP 400 with invalid_json.

Date: 2026-09-23.

The test also confirmed that the Cognito authentication flow redirected users to the hosted login page.

Further authenticated testing was completed on 28 September. A tester completed the full signed-in workflow by signing up, logging in, saving a Bangkok prediction, retrieving the private prediction history and asking the AI assistant about the saved estimate. Following the infrastructure rebuild on 6 October,

the public smoke workflow passed again and a direct deployed chat invocation using verified claims returned HTTP 200. The chatbot evaluation also passed all 16 tested scenarios locally and on the deployed Lambda.

6.3 Security Control Test

Security testing verified the private frontend boundary, direct API access controls and least-privilege behaviour.

Setup: Private S3 bucket with CloudFront Origin Access Control, API Gateway CORS restrictions, Cognito and JWT authorisation.

Test: S3 public-access settings and direct object access were checked, followed by API requests with and without valid authentication and approved origins.

Expected: Direct S3 access is denied, only the approved frontend origin is accepted, and protected API routes reject unauthenticated requests.

Result: Passed. All four S3 Block Public Access settings were enabled, direct S3 access returned HTTP 403, API Gateway CORS accepted the deployed CloudFront origin, and protected routes returned HTTP 401 without a valid token.

Date: 2026-09-23.

IAM testing confirmed that required application operations were permitted while unrelated actions, including raw S3 access and DynamoDB table scans, were denied. Deployed tests also used synthetic identities to verify owner-versus-other-user partition isolation; the test records were removed afterwards. A complete two-browser test using two Cognito accounts was not performed. ECR image scanning was performed, but its findings require separate evidence review, so the report makes no claim that the deployment is vulnerability-free.

6.4 Data/AI Validation Test

The data/AI tests validated the source dataset, preprocessing pipeline and model selection.

Setup: Supplied Listings and Reviews datasets, Python 3.11, pinned dependencies and the project profiling/training scripts.

Test: The profiling script was run followed by model training and evaluation.

Expected: The dataset profile is generated, the selected model and preprocessing pipeline are exported, and the reported evaluation metrics are reproduced.

Result: Passed for the reported data/model workflow. Profiling confirmed the dataset size, missing data and the absence of review text (Section 5.2). Histogram Gradient Boosting was selected on the supported held-out scope (metrics in Section 5.5).

Date: 2026-09-23.

A full-data replay on 6 October reproduced the stored dataset hash, train/test split, candidate model results, selected model and model hash. The offline preflight also completed successfully with all 47 unit tests passing.

6.5 Scalability/Resilience/Recovery Test

Resilience testing examined controlled overload and recovery without claiming unlimited scalability.

Setup: API Gateway, Lambda, DynamoDB point-in-time recovery and the development account concurrency limits.

Test: Bounded concurrent health and prediction requests were generated, followed by a quota-adjusted retest.

Expected: Excess demand should be controlled through HTTP 429 throttling rather than uncontrolled backend failure, while backup controls should support recovery.

Passed for the recorded bounded workloads. The quota-adjusted health and prediction tests returned a mixture of HTTP 200 and controlled HTTP 429 responses, with no recorded HTTP 5xx responses. Because the exact concurrency, request rate and effective throttle limit were not retained in the report evidence available for this review, the results are not presented as a capacity benchmark.

Dates: 2026-09-23 and 2026-09-29.

DynamoDB continuous backups and point-in-time recovery were enabled. On 9 October, a non-destructive restore created a separate temporary table, which was checked and then deleted. Application cutover to a restored table was not exercised, and cross-region recovery remains future work. The bounded 4 October health test produced only HTTP 200 and controlled HTTP 429 responses, with no recorded HTTP 5xx responses. A separate JMeter test exercised the CloudFront-hosted frontend with 25 simulated visitors over 45 seconds and recorded no failed requests. Exact response counts and latency values should be taken from the raw evidence file rather than duplicated in the report.

6.6 Monitoring and Operations

Operational monitoring uses CloudWatch logs, metrics, alarms and dashboards, with SNS for notifications.

Health Lambda invocations generated health_check events with a healthy status.
Prediction logs recorded model version and runtime metrics without recording listing inputs.
Glue and Athena recorded processing duration and data scanned.
An alarm test successfully entered ALARM, triggered SNS notification and returned to OK. The rebuilt stack's SNS subscription was confirmed, but delivery of a fresh alarm email to that rebuilt subscription was not retested.

Performance tuning also showed that increasing Lambda memory from **1 GB to 2 GB** reduced cold-start latency from approximately **13.8 seconds to 3.0–3.3 seconds**, while warm requests remained in the tens of milliseconds.

6.7 Failed Tests and Improvements

Issue	Cause	Improvement / Outcome
CloudWatch alarm failure	KMS policy did not allow CloudWatch to publish to SNS.	A scoped customer-managed key fixed the policy issue; the alarm then triggered SNS and returned to OK.
Excessive-load HTTP 503	Account-wide Lambda concurrency quota was exceeded.	The quota was increased and the test repeated, producing controlled HTTP 429 throttling with no 5xx responses.

7. Cost, Sustainability and Operational Considerations

7.1 Cost Considerations

Main cost drivers are AWS Lambda execution, Amazon API Gateway requests, Amazon CloudFront data transfer, Amazon S3 storage, Amazon Athena queries, interface VPC endpoints and Amazon Bedrock usage. Serverless compute reduces idle capacity but does not guarantee a lower total cost; interface VPC endpoints add fixed hourly charges even at low traffic. Parquet storage, Athena scan limits, result expiry and retention controls reduce avoidable processing and storage. The saved AWS Pricing Calculator workload estimate is approximately USD 72.92 per continuously deployed month before discounts and tax under the documented low-traffic assumptions. This is a planning estimate, not a record of actual charges; the two-AZ interface endpoint hours dominate the estimate.

7.2 Sustainability

Serverless compute avoids permanently running application servers, while CloudFront caching and Parquet reduce repeated processing.

7.3 Operational Requirements

Terraform manages infrastructure and GitHub Actions supports testing/deployment; CloudWatch and SNS provide monitoring and alerts.

DynamoDB point-in-time recovery and S3 protection support data protection. Operations require periodic review of performance, permissions, quotas, storage, query costs and AI usage.

7.4 Limitations and Future Improvements

The platform is not yet demonstrated at production scale. Key risks are usage costs, service quotas, cold starts, and dependence on multiple AWS services.

Future work should complete a full two-browser Cognito isolation journey, exercise application cutover to a restored DynamoDB table, evaluate cross-region recovery, add automated cost monitoring, broaden controlled load testing, and retrain the model with newer market data. The ECR findings were patched and rescanned clean at that point in time, bounded load profiles were recorded, and a non-destructive DynamoDB restore was completed on 9 October. Authenticated history, AI/chat and candidate-model comparisons are already implemented and tested, so they are not listed as future work.

8. Team Contribution, Ethical Considerations and Reflection

8.1 Team Contributions

Lum Jia Jun - Final testing of all the functions as well as helping Nixon with cloud deployment
Nixon Lee Disheng - Creating the diagrams and planning the infrastructure, writing the Terraform files and setting up the cloud database, as well as all the AWS services used in the project.

Madugula Adheesh - Cleaning the data, training, and creating the AI model, as well as testing it

Leow Yi Hao Ignatius - Creating the frontend for the website as well as linking it to AWS

Wong Zhen Ho Brendan - Documentation and minor changes to the model and frontend

8.2 Ethical and Responsible Practice

Ethical safeguards include data minimisation, authenticated access, and qualified model outputs. Cognito provides the verified identity used to partition prediction history, while logs exclude listing inputs. Bounded AI prompts and outputs reduce data exposure and cost abuse. The CC0 dataset still requires attribution. Because the data is historical and cross-sectional, prices use different currencies, and the model retains substantial error, predictions are presented as decision-support estimates rather than valuations or financial advice.

8.3 AI-Assisted Development

The team used OpenAI Codex for repository inspection, implementation support, Terraform, testing, documentation and deployment verification; Anthropic Claude Code for the signed-in estimator CORS fix, grounded chatbot improvements, verification support and architecture-diagram work; OpenAI ChatGPT for contribution and submission wording; Astra for an independent clean-Windows review; and OpenAI image generation for the decorative frontend and Cognito images. Amazon Bedrock Claude Haiku 4.5 powers the bounded authenticated chat feature. The team remained responsible for reviewing all generated work and verifying claims through tests, source review, Terraform plans, AWS CLI output and live application checks.

8.4 Reflection

Deployment showed that architecture claims require evidence. Testing exposed hidden dependencies, a KMS policy error, HTTP 503s and cold-start latency, prompting targeted fixes.

Appendix A – Test Summary

This table summarises the four compulsory test categories.

Test	Objective	Setup / Command	Expected	Actual	Evidence Path
Functional workflow	Verify the deployed frontend, API health endpoint, price prediction workflow and protected routes.	Terraform deployment in ap-southeast-1; tests/smoke_api.ps1 and live HTTP requests.	Frontend/health return HTTP 200; valid prediction returns a numeric estimate; malformed JSON returns HTTP 400; protected routes reject unauthenticated requests.	Passed. Frontend and health returned HTTP 200; live model returned 65.98 EUR; malformed JSON returned HTTP 400; protected routes returned HTTP 401.	evidence/test-functional.md, tests/smoke_api.ps1
Security control	Verify S3 access protection, CORS, input validation, authentication and least-privilege controls.	AWS CLI, Python unit tests, tests/smoke_api.ps1, Terraform IAM configuration.	S3 public access disabled; direct S3 access returns HTTP 403; invalid requests are rejected; protected routes require JWT; unnecessary IAM actions are denied.	Passed. S3 direct access returned HTTP 403; all four public-access settings were enabled; invalid requests returned HTTP 400; protected routes returned HTTP 401; selected unrelated IAM actions were denied.	evidence/test-security.md, tests/test_prediction.py, tests/test_history.py
Data/AI validation	Validate the dataset, preprocessing, model selection and reproducibility of the final model.	profile_data.py, train_model.py, verify_full_model.py and a deterministic test sample.	Dataset profile is generated; candidate models are compared; final model is selected using median city-normalised MAE; training results are reproducible.	Passed. Dataset contained 279,712 listings and 5,373,143 reviews. Histogram Gradient Boosting achieved 0.592 median city-normalised MAE and the full-data replay reproduced all seven recorded comparisons exactly.	evidence/test-data-ai.md, analytics/artifacts/model_evaluation.json, tests/verify_full_model.py
Scalability / resilience / recovery	Verify controlled overload handling and availability of	Bounded 100-request burst; CloudWatch	Requests should return HTTP 200 or controlled HTTP 429 without backend	Passed for the recorded bounded test: responses were HTTP 200 or	evidence/load-test-2026-10-04.txt, evidence/test-r

	recovery mechanisms.	monitoring; DynamoDB PITR check.	5xx/Lambda errors; PITR should be enabled.	controlled HTTP 429, with no recorded API 5xx or Lambda errors. DynamoDB point-in-time recovery was enabled; cross-region recovery was neither deployed nor tested. See the raw evidence file for exact response counts.	esilience.md, src/infrastructure/data.tf
--	----------------------	----------------------------------	--	--	--

Appendix B – Evidence Index

Evidence ID / Filename	What it proves
evidence/test-functional.md	Functional testing of the frontend, health endpoint, live prediction, authentication entry point and protected routes.
evidence/test-security.md	S3 access protection, CORS, input validation, authentication, history isolation, IAM permissions and security controls.
evidence/test-data-ai.md	Dataset profiling, model comparison, model evaluation, analytics pipeline and reproducibility checks.
analytics/artifacts/model_evaluation.json	Final model evaluation metrics and comparison of candidate models.
tests/verify_full_model.py	Full-dataset deterministic replay confirming the training pipeline reproduces the recorded model results.
evidence/load-test-2026-10-04.txt	Raw bounded load-test results showing controlled HTTP 429 throttling and no recorded backend errors.
evidence/test-resilience.md	Resilience testing, API throttling, DynamoDB PITR and recovery limitations.
evidence/cloud-verification-2026-10-04.md	Deployed infrastructure verification, Terraform state, service controls, alarms and application smoke checks.

Appendix C – References

Bhat, S. A. (n.d.). Airbnb Listings & Reviews [Data set]. Kaggle.

<https://www.kaggle.com/datasets/mysarahmadbhat/airbnb-listings-reviews> (CC0: Public Domain).

Amazon Web Services. (n.d.). Using PKCE in authorization code grants. Amazon Cognito Developer Guide.

<https://docs.aws.amazon.com/cognito/latest/developerguide/using-pkce-in-authorization-code.html>

Amazon Web Services. (n.d.). Restrict access to files. Amazon CloudFront Developer Guide.
<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-overview.html>

Amazon Web Services. (n.d.). Point-in-time backups for DynamoDB. Amazon DynamoDB Developer Guide.
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Point-in-time-recovery.html>

scikit-learn developers. (n.d.). HistGradientBoostingRegressor. scikit-learn documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>

Appendix D – Monitoring Evidence

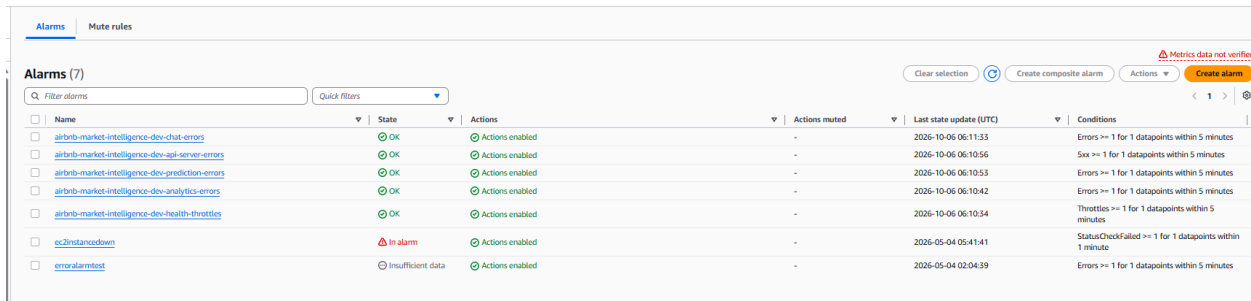


Figure D1. Amazon CloudWatch alarm dashboard showing configured alarm states and notification actions.

Appendix E - Final Submission Evidence Index

This index matches the finalized repository report and submission package

Appendix	Contents	Evidence in the repository
A	Full-size architecture diagram and editable source	evidence/architecture-current.png; evidence/architecture-full.dra wio
B	Cost estimate and assumptions (estimate, not actual charges)	evidence/aws-pricing-calculat or-2026-10-06.md; evidence/aws-pricing-calculat or-estimate-2026-10-06.csv; evidence/cost-estimate.md
C	Monitoring, five alarms, dashboard, verifier output and redacted SNS alarm emails	evidence/monitoring.md; evidence/cloud-verification-20 26-10-09.md; evidence/appendix-c-sns-alar m-email-2026-10-03-redacted .png; evidence/appendix-c-alarm-e mail-insufficient-to-ok-2026-1 0-03-redacted.png; evidence/appendix-c-alarm-e mail-ok-to-alarm-2026-10-03- redacted.png
D	Functional, security, data/AI, scalability and resilience tests	evidence/test-functional.md; evidence/test-security.md; evidence/test-data-ai.md; evidence/test-resilience.md

E	Latest live stack, recovery, chatbot, security, load and DynamoDB restore tests	evidence/live-stack-test-2026-10-09.md; evidence/chatbot-evaluation-deployed-2026-10-07.json; evidence/manual-browser-verification-2026-10-07.md
F	Threat-control map, OWASP review and Terraform static-scan triage	evidence/threat-control-map.md; evidence/owasp-top-10-2025-review.md; evidence/iac-scan-triage-2026-10-10.md
G	Model evaluation, data dictionary and pipeline	analytics/artifacts/model_evaluation.json; data/DATA_DICTIONARY.md; evidence/data-pipeline.md
H	CI/CD results and the 9 October review/fix record	evidence/ci-cd.md; evidence/github-security-runs-2026-10-06.md; evidence/sanity-review-fixes-2026-10-09.md
I	AI-use and team-contribution declarations	AI_USE_DECLARATION.md; TEAM_CONTRIBUTIONS.md